

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Paleru Dharani Govardhan	Reg. No.:	192525280
Section / Batch:	2025/SLOT-C	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I PALERU DHARANI GOVARDHAN / 192525280 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: DHARANI

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT TOOL 3

Comparative Analysis

Comparative Analysis of Dynamic Programming and Greedy Algorithms for Optimization Problems

Course Code	CSA06
Course Title	Design and Analysis of Algorithms
CO Assessed	CO4 — Articulation (BL4, BL5)
CO4 Statement	Solve optimization problems using dynamic programming and greedy strategies.
Topic	Comparative Analysis of Dynamic Programming and Greedy Algorithms for Optimization Problems
Student Name	Paleru Dharani Govardhan
Register No.	192525280
Section / Batch	2025/SLOT-C
Date	25-08-26

1. Introduction and Objectives

Optimization is the process of systematically choosing, from a set of possible decisions, the one that produces the best measurable outcome. In algorithmic problem-solving, an optimization problem asks for a solution that either minimizes a cost (for example, distance travelled, time taken, or resources consumed) or maximizes a benefit (for example, profit, throughput, or coverage), while respecting a defined set of rules or limits.

1.1 Key Terminology

- **Objective function:** the quantity being optimized, expressed as a function of the decision variables (for example, total value carried, total distance travelled, or total time taken).
- **Feasible solution:** any assignment of decision variables that satisfies all the stated constraints, whether or not it is the best possible one.
- **Optimal solution:** the feasible solution that gives the best possible value of the objective function among all feasible solutions.
- **Decision variables:** the quantities that can be chosen or controlled, such as which items to select or which edges to include.
- **Constraints:** the rules that restrict which combinations of decisions are allowed, such as a weight limit or a time window.

1.2 Purpose of This Comparison

Two broad algorithmic strategies are commonly used to solve optimization problems on discrete structures: Dynamic Programming (DP) and Greedy algorithms. Both strategies build a solution incrementally, but they differ fundamentally in how they explore the space of possible decisions. DP considers multiple overlapping possibilities and stores intermediate results so that no subproblem is solved twice, guaranteeing global optimality whenever the recurrence is correctly formulated. Greedy algorithms instead commit to the locally best decision at every step and never revisit that decision, which is far cheaper computationally but is only guaranteed to be globally optimal when the problem exhibits a specific structural property known as the greedy-choice property.

This working draft develops a structured, evidence-based comparison of the two strategies: their principles, representative problems, worked numerical examples, complexity behaviour, correctness arguments, and practical trade-offs, so that an informed choice can be made between them for a given optimization problem.

1.3 Relevance to CO4

CO4 requires the ability to solve optimization problems using dynamic programming and greedy strategies, assessed at Bloom's Taxonomy levels BL4 (Analysis) and BL5 (Evaluation). This draft is organized to support both levels: BL4 is addressed through the decomposition of representative problems into their recurrence relations, greedy rules, and complexity structures (Sections 3 to 10); BL5 is addressed through the critical evaluation, correctness reasoning, and algorithm-selection discussion (Sections 11 to 13), where you are asked to weigh strengths and limitations and justify a choice of strategy rather than simply describe it.

2. Optimization Problems

An optimization problem is formally defined by a search space of candidate solutions, an objective function that assigns a numerical value to each candidate, and a set of constraints that determine which candidates are feasible. Solving the problem means finding a feasible candidate whose objective value is best, either smallest (minimization) or largest (maximization).

2.1 Minimization and Maximization

- Minimization problems seek the lowest possible value of a cost function, for example minimizing total travel distance in a routing problem or minimizing total waiting time in a scheduling problem.
- Maximization problems seek the highest possible value of a benefit function, for example maximizing total profit in a knapsack problem or maximizing the number of activities completed in a scheduling problem.
- Many problems can be converted between the two forms by negating the objective function, so the underlying algorithmic techniques apply to both.

2.2 Constraints and Feasibility

Constraints define the boundary of allowed solutions. Typical constraint types include capacity constraints (a knapsack cannot exceed its weight limit), precedence constraints (a task cannot start before its prerequisite finishes), and mutual-exclusion constraints (two activities cannot use the same resource at overlapping times). A solution that violates any constraint is infeasible and is excluded from consideration regardless of its objective value.

2.3 Local Optimum vs Global Optimum

- A local optimum is a solution that is better than all feasible solutions in its immediate neighbourhood, but not necessarily better than every feasible solution overall.
- A global optimum is a solution that is better than or equal to every other feasible solution in the entire search space.
- Greedy algorithms can become trapped at a local optimum if applied to problems that lack the greedy-choice property, whereas dynamic programming systematically evaluates enough of the space to guarantee a global optimum.

2.4 Combinatorial Optimization

Combinatorial optimization problems involve selecting, ordering, or grouping discrete objects from a finite set, subject to constraints, such as choosing a subset of items, an ordering of tasks, or a spanning structure over a graph. The number of possible combinations typically grows exponentially with the input size, which is why brute-force enumeration quickly becomes computationally infeasible and structured techniques such as DP and Greedy are required.

2.5 Real-World Examples

- Logistics: selecting delivery routes that minimize total distance or fuel cost while meeting time windows.
- Finance: selecting a subset of investments that maximizes expected return within a risk or budget limit.
- Computing: scheduling processes on a CPU to minimize average waiting time or maximize throughput.
- Telecommunications: designing a minimum-cost network that connects all required nodes.
- Data compression: assigning shorter binary codes to more frequent symbols to minimize encoded file size.

3. Dynamic Programming

Dynamic Programming (DP) is a technique for solving optimization problems by breaking them into smaller overlapping subproblems, solving each distinct subproblem only once, and storing its result for reuse. DP is applicable whenever a problem exhibits two structural properties: optimal substructure and overlapping subproblems.

3.1 Optimal Substructure

A problem has optimal substructure if an optimal solution to the problem can be constructed from optimal solutions to its subproblems. This allows a large problem to be solved by combining the best answers to smaller instances of the same problem, rather than having to search the entire solution space directly.

3.2 Overlapping Subproblems

A problem has overlapping subproblems if a recursive solution would solve the same subproblem repeatedly. DP exploits this by computing each subproblem's answer exactly once and storing it, which converts an exponential number of recursive calls into a polynomial number of table entries.

3.3 Recurrence Relations

The core of any DP solution is a recurrence relation that expresses the optimal value of a problem instance in terms of the optimal values of smaller instances, together with one or more base cases that terminate the recursion. For example, a general knapsack-style recurrence takes the form:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}(i)] + \text{value}(i)) \text{ if } \text{weight}(i) \leq w$$

3.4 Memoization (Top-Down)

- Starts from the original problem and recurses into subproblems as needed.
- Uses a cache (array, hash map) to store the result of each subproblem the first time it is computed.
- On repeated calls to the same subproblem, the cached value is returned instantly instead of recomputing.
- Naturally mirrors the recursive definition of the problem, which can make the code easier to derive from the recurrence.

3.5 Tabulation (Bottom-Up)

- Starts from the base cases and iteratively fills a table up to the size of the original problem.
- Avoids recursion overhead and stack-depth limitations, which matters for large inputs.
- Requires the subproblems to be processed in a valid dependency order, smallest first.
- Is generally preferred in production code for its predictable memory access pattern and lower constant-factor overhead.

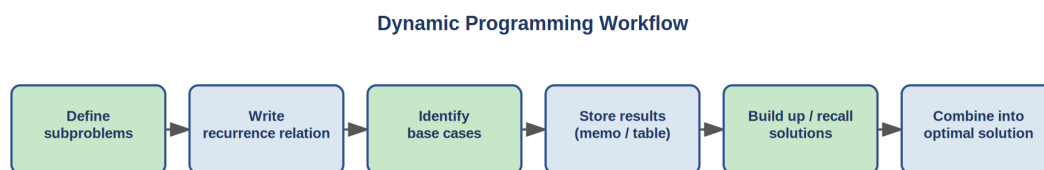


Figure 1. Dynamic Programming Workflow

The DP process moves from problem decomposition through recurrence design, base cases, and storage to a combined optimal solution.

4. Greedy Algorithms

A Greedy algorithm builds a solution incrementally by making, at each step, the choice that looks best according to some local criterion, without reconsidering previous choices. Greedy algorithms are simple and efficient, but they only produce a globally optimal solution when the problem satisfies specific structural conditions.

4.1 The Greedy-Choice Property

A problem has the greedy-choice property if a globally optimal solution can always be reached by making a locally optimal (greedy) choice first, and then solving the remaining subproblem optimally. This is the condition that must be proven, not merely assumed, before a greedy strategy can be trusted to give the correct answer.

4.2 Local Decisions and Feasibility

- At each step the algorithm evaluates the available candidates using a fixed ranking or selection rule (for example, earliest finish time, highest ratio, or lowest edge weight).
- The best-ranked candidate is chosen only if it keeps the partial solution feasible.
- Once a candidate is chosen or rejected, that decision is final and is never revisited, which is what makes greedy algorithms fast but also what makes them risky when the greedy-choice property does not hold.

4.3 Optimal Substructure in Greedy Problems

Like DP, correct greedy algorithms also rely on optimal substructure: after the greedy choice is made, the remaining problem must itself be an instance of the same type of problem, so that greedy reasoning can be applied recursively (in principle, though usually implemented iteratively) until the full solution is built.

4.4 Correctness Requirements

- The greedy-choice property must be proven for the specific problem, typically using an exchange argument that shows any optimal solution can be transformed to include the greedy choice without any loss in objective value.
- Optimal substructure must also hold, so that solving the remainder greedily continues to be valid.
- A ranking or ordering criterion must exist that can be computed cheaply and applied consistently.

4.5 Cases Where Greedy Fails

The classic counter-example is the 0/1 Knapsack problem: choosing items purely by descending value-to-weight ratio can leave unused capacity that would have been better filled by a different combination, because items cannot be split. In such cases the greedy-choice property does not hold and a DP formulation is required to guarantee the optimal answer; the Fractional Knapsack problem, by contrast, allows items to be split and does satisfy the greedy-choice property, so ratio-based greedy selection is provably optimal there (see Section 7).

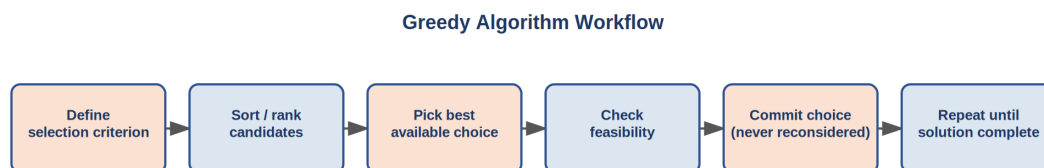


Figure 2. Greedy Algorithm Workflow

The greedy process ranks candidates, commits to the best feasible choice at each step, and never reconsiders a prior decision.

5. Representative Problems

The table below lists widely studied optimization problems solved with each strategy, together with the core idea that makes the strategy applicable. These problems recur throughout algorithm courses because each one cleanly

illustrates either optimal-substructure-with-overlap (favouring DP) or a provable greedy-choice property (favouring Greedy).

5.1 Dynamic Programming Problems

Problem	Core Idea	Typical Complexity
0/1 Knapsack	Choose items to maximize value within a weight limit; items cannot be split.	$O(nW)$
Longest Common Subsequence (LCS)	Find the longest sequence common to two strings, built from matching/skipping characters.	$O(mn)$
Matrix Chain Multiplication	Choose the parenthesization of a matrix product that minimizes scalar multiplications.	$O(n^3)$
Coin Change (minimum coins)	Find the minimum number of coins that sum to a target amount.	$O(n \times \text{amount})$
Rod Cutting	Cut a rod into pieces to maximize total revenue from piece prices.	$O(n^2)$

Table 1. Representative dynamic programming problems and their complexity.

5.2 Greedy Problems

Problem	Core Idea	Typical Complexity
Fractional Knapsack	Fill the knapsack by descending value/weight ratio; items may be split.	$O(n \log n)$
Activity Selection	Select the maximum number of non-overlapping activities by earliest finish time.	$O(n \log n)$
Huffman Coding	Build a minimum-cost prefix code by repeatedly merging the two least-frequent nodes.	$O(n \log n)$
Job Sequencing with Deadlines	Schedule jobs to maximize profit, each job assigned to the latest free slot before its deadline.	$O(n \log n)$
Prim's Algorithm (MST)	Grow a minimum spanning tree by always adding the cheapest edge that connects a new vertex.	$O(E \log V)$
Kruskal's Algorithm (MST)	Build a minimum spanning tree by adding the cheapest edge that does not form a cycle.	$O(E \log E)$

Table 2. Representative greedy problems and their complexity.

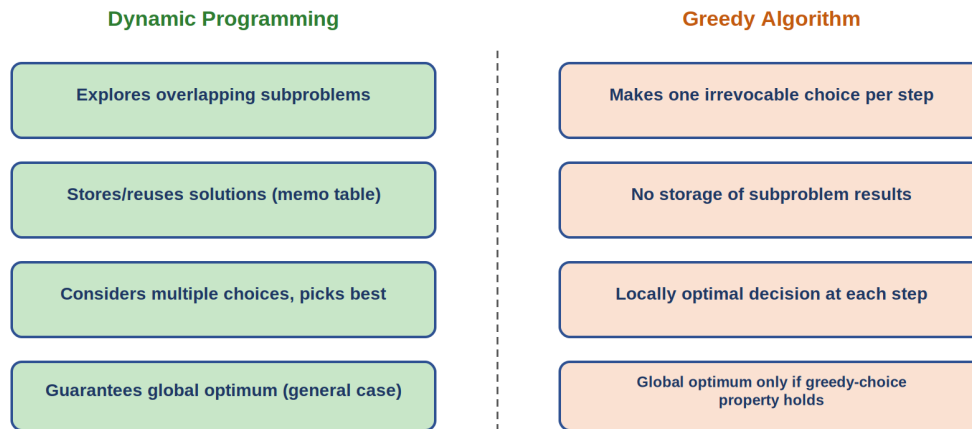


Figure 3. Conceptual Comparison of Dynamic Programming and Greedy Algorithms

DP explores and stores overlapping subproblem results to guarantee optimality; Greedy commits to a single locally optimal choice at each step.

6. 0/1 Knapsack — Worked Example

6.1 Problem Definition

Given n items, each with a weight and a value, and a knapsack of capacity W , choose a subset of items whose total weight does not exceed W and whose total value is maximized. Each item may be taken at most once (it cannot be split), which is what distinguishes this from the Fractional Knapsack problem in Section 7.

6.2 Numerical Example

Consider four items and a knapsack capacity of $W = 8$:

Item	Weight	Value
A	2	3
B	3	4
C	4	5
D	5	6

Table 3. Item weights and values for the 0/1 Knapsack worked example ($W = 8$).

6.3 Dynamic Programming Recurrence

Let $dp[i][w]$ be the maximum value achievable using the first i items with capacity w . The recurrence is:

$$\begin{aligned}
 dp[i][w] &= dp[i-1][w], \text{ if } \text{weight}(i) > w \\
 dp[i][w] &= \max(dp[i-1][w], dp[i-1][w - \text{weight}(i)] + \text{value}(i)), \text{ if } \text{weight}(i) \leq w \\
 \text{Base case: } dp[0][w] &= 0 \text{ for all } w \text{ (no items considered means zero value).}
 \end{aligned}$$

6.4 Complete DP Table

$i \setminus w$	0	1	2	3	4	5	6	7	8
0 (none)	0	0	0	0	0	0	0	0	0

I \ w	0	1	2	3	4	5	6	7	8
1 (A: w2,v3)	0	0	3	3	3	3	3	3	3
2 (B: w3,v4)	0	0	3	4	4	7	7	7	7
3 (C: w4,v5)	0	0	3	4	5	7	8	9	9
4 (D: w5,v6)	0	0	3	4	5	7	8	9	10

Table 4. Complete 0/1 Knapsack DP table; $dp[4][8] = 10$ is the optimal value.

6.5 Step-by-Step Calculation and Optimal Solution

- Row 1 (item A, weight 2, value 3): for $w < 2$ the value stays 0; for $w \geq 2$, $dp[1][w] = 3$.
- Row 2 (item B, weight 3, value 4): at $w = 5$, $dp[2][5] = \max(dp[1][5], dp[1][2] + 4) = \max(3, 3+4) = 7$, so both A and B fit and are both taken.
- Row 3 (item C, weight 4, value 5): at $w = 6$, $dp[3][6] = \max(dp[2][6], dp[2][2] + 5) = \max(7, 3+5) = 8$; taking C instead of B at this capacity is better.
- Row 4 (item D, weight 5, value 6): at $w = 8$, $dp[4][8] = \max(dp[3][8], dp[3][3] + 6) = \max(9, 4+6) = 10$, the table's final and optimal value.
- Tracing back from $dp[4][8] = 10$: it came from $dp[3][3] + 6$, so item D is taken; $dp[3][3] = 4$ came from $dp[2][3]$, so item C is not taken; $dp[2][3] = 4$ came from $dp[1][0] + 4$, so item B is taken; $dp[1][0] = 0$ means item A is not taken.
- Optimal solution: items B and D, total weight $3 + 5 = 8$ (exactly fills the capacity), total value $4 + 6 = 10$.

6.6 Correctness Explanation

The recurrence is correct because, for each item I and capacity w , there are only two possibilities: item I is excluded, in which case the best value is whatever the first $i-1$ items could achieve with the same capacity, or item I is included, in which case the best value is item I 's value plus the best value the first $i-1$ items could achieve with the remaining capacity $w - \text{weight}(i)$. Because $dp[i-1][*]$ is guaranteed to already hold the optimal value for every smaller capacity (by the same argument applied inductively from the base case $dp[0][*] = 0$), taking the maximum of these two possibilities is guaranteed to produce the true optimal value $dp[i][w]$.

6.7 Time and Space Complexity

- Time complexity: $O(nW)$, since the table has $(n+1) \times (W+1)$ entries and each entry is computed in $O(1)$ time from two previously computed entries.
- Space complexity: $O(nW)$ for the full table, reducible to $O(W)$ if only the final optimal value (not the selected items) is required, by keeping just the previous row.
- This is described as pseudo-polynomial time because it is polynomial in the numeric value of W , not in the number of bits needed to represent W ; for very large W this can still be impractical.

7. Fractional Knapsack — Worked Example

The Fractional Knapsack problem uses the same setup as 0/1 Knapsack — items with weights and values, and a capacity limit — except that items may be split, so any fraction of an item may be taken. This relaxation is exactly what allows a greedy strategy to guarantee optimality.

7.1 Numerical Example (Same Items as Section 6)

Using the same four items and capacity $W = 8$ as the 0/1 Knapsack example, first compute the value-to-weight ratio of each item:

Item	Weight	Value	Value / Weight Ratio
A	2	3	1.50
B	3	4	1.33
C	4	5	1.25
D	5	6	1.20

Table 5. Value-to-weight ratios used to rank items for the greedy selection.

7.2 Sorting by Ratio

Sorting items in descending order of ratio gives the selection order: A (1.50) → B (1.33) → C (1.25) → D (1.20).

7.3 Greedy Selections

- Take item A fully: weight used 2, value gained 3, remaining capacity 6.
- Take item B fully: weight used 3, value gained 4, remaining capacity 3.
- Item C has weight 4, but only 3 capacity remains, so take a $\frac{3}{4}$ fraction of C: value gained = $5 \times 0.75 = 3.75$, remaining capacity 0.
- Item D is not taken; the knapsack is full.

7.4 Final Solution

Total weight used: $2 + 3 + 3 = 8$ (exactly the capacity). Total value: $3 + 4 + 3.75 = 10.75$.

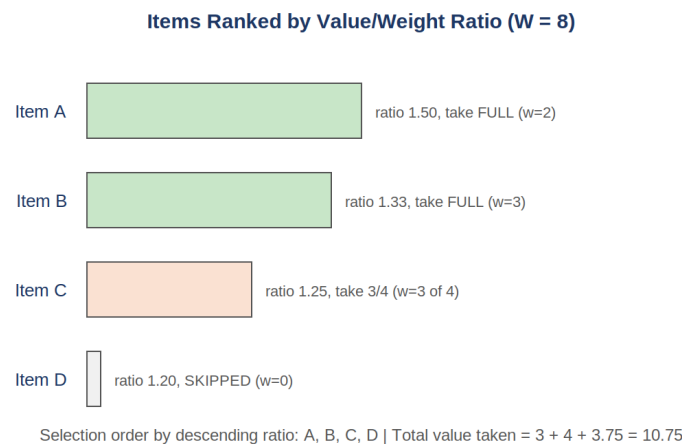


Figure 4. Fractional Knapsack Greedy Selection

Items are ranked by value/weight ratio; the algorithm fully takes the top-ranked items and takes only a partial fraction of the item that would otherwise overflow capacity.

7.5 Correctness Reasoning

The greedy-choice property holds here because, per unit of weight, no other selection can produce more value than always filling capacity with the currently highest-ratio item available. If an optimal solution took less than the maximum possible amount of a higher-ratio item while taking a positive amount of a lower-ratio item, swapping weight from the lower-ratio item to the higher-ratio item (which is always possible since items are divisible) would strictly increase total value without exceeding capacity — an exchange argument that proves any optimal solution must match the greedy selection.

7.6 Complexity

- Sorting the items by ratio dominates the running time: $O(n \log n)$.
- The single pass that fills the knapsack after sorting takes $O(n)$ time.
- Overall time complexity: $O(n \log n)$; space complexity: $O(n)$ for storing the ratios and ordering, or $O(1)$ additional space if sorting is performed in place.

7.7 Direct Comparison with 0/1 Knapsack

Aspect	0/1 Knapsack (Section 6)	Fractional Knapsack (this page)
Items divisible?	No	Yes
Strategy used	Dynamic Programming	Greedy (ratio-based)
Optimal value (same items, $W = 8$)	10	10.75
Time complexity	$O(nW)$	$O(n \log n)$
Greedy-choice property	Does not hold	Holds (provable by exchange argument)

Table 6. Direct comparison of 0/1 Knapsack and Fractional Knapsack on the identical item set.

The fractional version achieves a strictly higher value (10.75 versus 10) using the same items and capacity, because splitting item C lets the knapsack be filled with no wasted capacity. This single numerical contrast is a concrete illustration of a broader principle developed further in Section 9: relaxing a problem's constraints can change which algorithmic strategy is provably optimal.

8. Activity Selection — Worked Example

The Activity Selection problem asks: given a set of activities, each with a start time and a finish time, and a single shared resource (for example, one meeting room), select the maximum possible number of activities such that no two selected activities overlap in time.

8.1 Activity Table

Activity	Start	Finish
A1	1	4
A2	3	5
A3	0	6
A4	5	7
A5	3	9
A6	5	9
A7	6	10
A8	8	11

Activity	Start	Finish
A9	8	12
A10	2	14
A11	12	16

Table 7. Eleven candidate activities with start and finish times.

8.2 Finish-Time Sorting

The greedy strategy sorts all activities by finish time in ascending order: A1(4), A2(5), A3(6), A4(7), A5(9), A6(9), A7(10), A8(11), A9(12), A10(14), A11(16). Sorting by finish time, rather than by start time or duration, is the specific choice that makes the greedy-choice property hold for this problem.

8.3 Selection Steps

- Select A1 (1–4), the activity with the earliest finish time. Last finish time is now 4.
- A2 (3–5) and A3 (0–6) both start before time 4, so both overlap with A1 and are rejected.
- Select A4 (5–7), the next activity whose start time (5) is not earlier than the last finish time (4). Last finish time is now 7.
- A5 (3–9), A6 (5–9), and A7 (6–10) all start before time 7 and are rejected.
- Select A8 (8–11), whose start time (8) is not earlier than 7. Last finish time is now 11.
- A9 (8–12) and A10 (2–14) start before time 11 and are rejected.
- Select A11 (12–16), whose start time (12) is not earlier than 11. Last finish time is now 16.
- Final selected set: { A1, A4, A8, A11 } — four non-overlapping activities, which is the maximum possible for this instance.

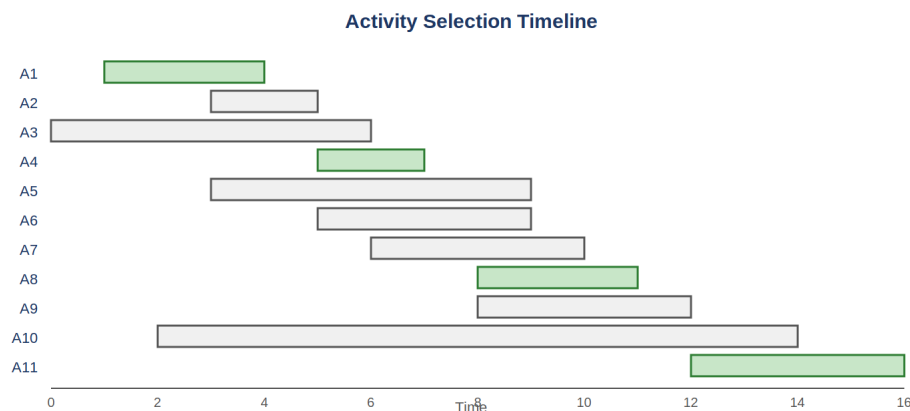


Figure 5. Activity Selection Timeline

Selected activities (highlighted) never overlap in time; each is chosen because its start time is not earlier than the finish time of the previously selected activity.

8.4 Correctness Reasoning

The greedy-choice property holds because choosing the activity with the earliest finish time always leaves the maximum possible remaining time for scheduling further activities. If an optimal solution did not include the earliest-finishing activity, it could be exchanged for it without reducing the count of selected activities and without causing any new overlap, since the earliest-finishing activity frees up resource time at least as early as any alternative first choice. Applying this exchange argument repeatedly to the remaining subproblem (activities

that start after the current last finish time) proves that the greedy selection is optimal at every step, and therefore overall.

8.5 Complexity

- Sorting by finish time: $O(n \log n)$.
- Single linear scan to select compatible activities: $O(n)$.
- Overall time complexity: $O(n \log n)$, dominated by the sort; space complexity: $O(n)$ to hold the sorted list, or $O(1)$ additional space if the input is already sorted.

8.6 Real-World Interpretation

This problem models any single shared resource that can serve only one request at a time, such as booking a single meeting room, scheduling a single machine to run one job at a time, or allocating a single runway for aircraft landing slots. The greedy earliest-finish-time rule generalizes directly to these settings whenever the objective is simply to maximize the number of requests served, with no notion of differing priority or value per request; if requests instead carry different weights or profits, the problem becomes Weighted Activity Selection, which loses the simple greedy-choice property and is typically solved with dynamic programming instead.

9. Dynamic Programming vs Greedy — Comprehensive Comparison

The table below consolidates the structural, computational, and practical differences between the two strategies discussed throughout Sections 3 to 8, drawing on the worked examples for concrete evidence.

Criterion	Dynamic Programming	Greedy Algorithm
Strategy	Explores and combines solutions to overlapping subproblems.	Makes one irrevocable locally optimal choice per step.
Decision process	Considers multiple options at each stage before committing.	Considers only the current best-ranked option.
Optimal substructure	Required.	Required.
Greedy-choice property	Not applicable / not required.	Required and must be proven.
Overlapping subproblems	Required (this is what DP exploits).	Not exploited; no subproblem reuse.
Memory usage	Higher — typically $O(n)$ to $O(nW)$ for tables.	Lower — typically $O(1)$ to $O(n)$.
Time complexity (typical)	Polynomial in input size and value range, e.g. $O(nW)$, $O(n^2)$, $O(n^3)$.	Usually $O(n \log n)$, dominated by sorting.
Correctness guarantee	Always optimal, given a correct recurrence.	Optimal only if greedy-choice property holds.
Exactness	Exact for all correctly modeled problems.	Exact only for a specific class of problems; otherwise approximate or wrong.
Scalability	Can become expensive for large state spaces (large n or large W).	Scales very well; usually the fastest option when applicable.
Implementation complexity	Moderate to high — requires defining states and recurrence carefully.	Low — usually a sort plus a single pass.
Typical applications	Knapsack, LCS, MCM, Coin Change, Rod Cutting.	Fractional Knapsack, Activity Selection, Huffman Coding, MST (Prim/Kruskal).

Criterion	Dynamic Programming	Greedy Algorithm
Main limitation	Memory and time cost grow with state-space size; recurrence design can be non-trivial.	Cannot be used safely without first proving the greedy-choice property; may silently give suboptimal results.

Table 8. Comprehensive comparison of Dynamic Programming and Greedy algorithms across structural and practical criteria.

9.1 Analytical Commentary

The comparison shows that the two strategies are not interchangeable alternatives to the same problem but are suited to structurally different problem classes. The determining question is not which strategy is “better” in general, but whether the specific problem at hand possesses the greedy-choice property. When it does, as in Fractional Knapsack and Activity Selection, greedy algorithms deliver the optimal answer with substantially lower time and memory cost than a DP formulation would require. When it does not, as in 0/1 Knapsack, a greedy strategy can produce a plausible-looking but provably suboptimal answer, and DP becomes necessary to guarantee correctness, at the cost of higher time and memory usage.

A second observation from the worked examples in Sections 6 and 7 is that constraint relaxation can change which strategy is appropriate for what otherwise looks like the same problem: allowing items to be split converted 0/1 Knapsack, which needs DP, into Fractional Knapsack, which is correctly and efficiently solved by a greedy rule. This suggests a practical diagnostic question for any new optimization problem: does relaxing or tightening a single constraint change whether a locally optimal choice remains globally optimal? If yes, that constraint is likely the reason DP is required rather than Greedy.

10. Complexity Analysis

10.1 Representative Problems and Their Complexities

Problem	Strategy	Time Complexity	Space Complexity
0/1 Knapsack	Dynamic Programming	$O(nW)$	$O(nW)$, reducible to $O(W)$
Fractional Knapsack	Greedy	$O(n \log n)$	$O(n)$
Longest Common Subsequence	Dynamic Programming	$O(mn)$	$O(mn)$, reducible to $O(\min(m,n))$
Matrix Chain Multiplication	Dynamic Programming	$O(n^3)$	$O(n^2)$
Coin Change (minimum coins)	Dynamic Programming	$O(n \times \text{amount})$	$O(\text{amount})$
Activity Selection	Greedy	$O(n \log n)$	$O(n)$
Huffman Coding	Greedy	$O(n \log n)$	$O(n)$
Prim's Algorithm (MST)	Greedy	$O(E \log V)$	$O(V + E)$
Kruskal's Algorithm (MST)	Greedy	$O(E \log E)$	$O(V)$

Table 9. Time and space complexity of representative Dynamic Programming and Greedy algorithms.

10.2 Time Complexity

DP algorithms on this list range from $O(nW)$, which depends on the numeric capacity value and is therefore pseudo-polynomial, to $O(n^3)$ for Matrix Chain Multiplication, which is fully polynomial in the number of

matrices. Greedy algorithms on this list are consistently $O(n \log n)$ or close to it, because their cost is almost entirely determined by an initial sort or a priority-queue-based selection process.

10.3 Space Complexity

DP space requirements are generally driven by the size of the table needed to store subproblem results, which for two-dimensional recurrences (Knapsack, LCS, MCM) scales with the product of the two dimensions unless a rolling-array optimization is used to keep only the most recently needed rows. Greedy algorithms typically need only linear space to hold the input and its sorted order, since no subproblem table is maintained.

10.4 Best/Worst Cases

- 0/1 Knapsack: the DP table is always fully computed regardless of the specific item values, so best-case and worst-case time complexity coincide at $O(nW)$.
- Activity Selection and Fractional Knapsack: the dominant cost is sorting, whose worst case is $O(n \log n)$ for comparison-based sorts; if the input is supplied pre-sorted, practical running time can approach $O(n)$, though the standard complexity bound is still stated as $O(n \log n)$.
- Huffman Coding: always requires $O(n \log n)$ due to the repeated extraction of the two smallest-frequency nodes from a priority queue, regardless of the symbol-frequency distribution.

10.5 Polynomial vs Exponential Behaviour

Both DP and Greedy strategies exist specifically to avoid the exponential time complexity of brute-force enumeration, which for an n -item selection problem would require examining up to 2^n subsets. DP achieves polynomial time by collapsing an exponential number of overlapping recursive calls into a polynomial number of distinct table entries; Greedy achieves polynomial time by never exploring alternative branches at all, provided doing so is safe (that is, provided the greedy-choice property holds).

10.6 Scalability

For very large inputs, the $O(n \log n)$ growth of typical greedy algorithms scales substantially better than the $O(nW)$ or $O(n^2)$ to $O(n^3)$ growth of typical DP algorithms, particularly when W or the problem's other numeric parameters are large relative to n . This is one of the main practical reasons to prefer a greedy formulation whenever the underlying problem structure permits it.

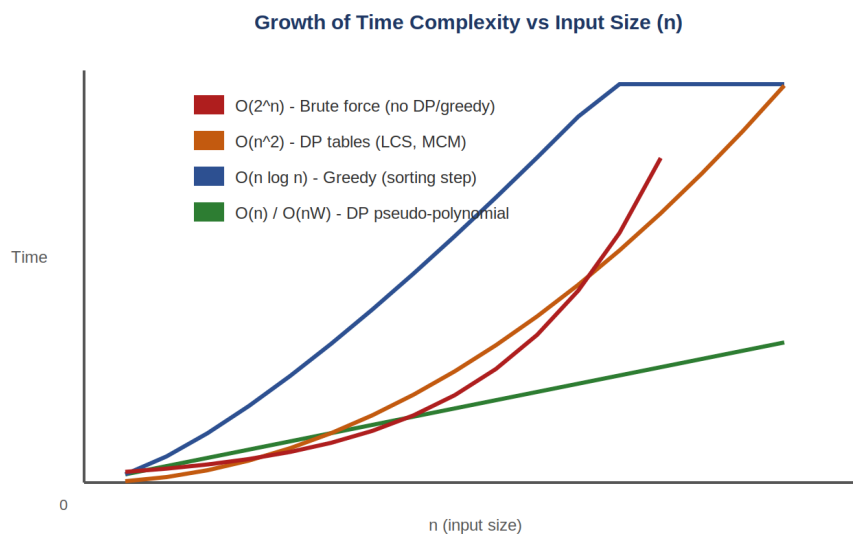


Figure 6. Illustrative Growth of Time Complexity vs Input Size

Schematic comparison of complexity classes relevant to this comparison; curves are illustrative of relative growth rates, not drawn from a specific benchmark.

11. Correctness

Proving that an algorithm is correct means showing that it always produces an optimal feasible solution for every valid input, not merely for the examples that happen to have been tested. DP and Greedy algorithms require different styles of correctness argument, reflecting their different structures.

11.1 Dynamic Programming Correctness

- **Recurrence:** the recurrence relation must account for every way an optimal solution to a subproblem could be formed from smaller subproblems, with no case omitted.
- **Base cases:** the smallest subproblems must be assigned correct, directly verifiable values (for example, $dp[0][w] = 0$ for the empty item set).
- **Optimal substructure:** it must be shown that an optimal solution to the full problem necessarily contains optimal solutions to the subproblems it depends on.
- **Induction-style reasoning:** assuming $dp[i-1][*]$ is correct for all capacities (the inductive hypothesis), the recurrence must be shown to compute a correct $dp[i][*]$ by considering all valid choices at step i and taking the best; since the base case is directly correct, induction extends correctness to every row up to $dp[n][W]$.

11.2 Greedy Correctness

- **Greedy-choice property:** it must be proven, not assumed, that some optimal solution begins with the greedy choice; assuming this without proof is the single most common error in applying greedy algorithms.
- **Exchange argument:** take an arbitrary optimal solution that does not start with the greedy choice, and show that it can be modified to include the greedy choice instead, without decreasing (and sometimes strictly increasing) the objective value; this shows the greedy choice is always at least as good as any alternative first choice.
- **Optimal substructure:** after the greedy choice is fixed, the remaining problem must be shown to be a smaller instance of the same problem, so that the same greedy reasoning can be applied again until the full solution is built.

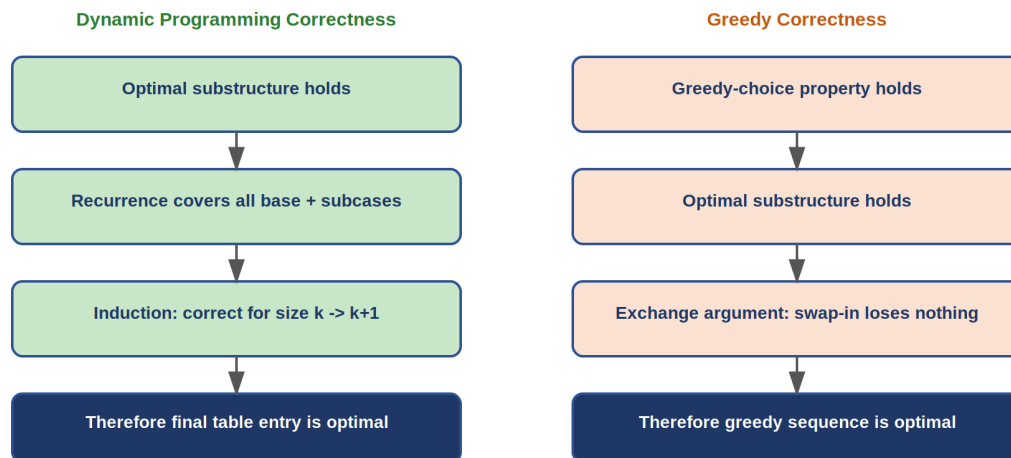


Figure 7. Correctness Frameworks for Dynamic Programming and Greedy Algorithms

DP correctness follows an inductive argument built on the recurrence and base cases; Greedy correctness follows an exchange argument built on the greedy-choice property.

11.3 Worked Correctness Note (Activity Selection)

As a concrete instance of the exchange argument described above: suppose an optimal solution to the Activity Selection example in Section 8 did not include A1 (the earliest-finishing activity) as its first activity. Since A1 finishes no later than any other activity's finish time, replacing whatever activity that optimal solution did choose first with A1 cannot cause any new overlap with the activities that follow, and leaves the count of selected activities unchanged. This shows an optimal solution containing A1 first always exists, which is exactly the greedy-choice property; the same argument then applies recursively to the remaining activities that start after A1 finishes.

12. Critical Evaluation

12.1 Dynamic Programming

Strengths:

- Guarantees a globally optimal solution whenever the recurrence correctly models the problem.
- Handles problems with overlapping subproblems efficiently by eliminating redundant recomputation.
- Applicable to a very wide class of optimization problems, including many where no greedy strategy exists at all.

Limitations:

- Time and space costs scale with the size of the state space, which can be large when a problem has many parameters (for example, multiple resource constraints).
- Designing a correct recurrence and identifying the right subproblem definition can be conceptually difficult and error-prone.
- Pseudo-polynomial complexities (such as $O(nW)$) can become impractical when numeric parameters like capacity are very large, even though n itself is small.

Typical Use Cases:

- Resource-constrained selection problems where items cannot be split (0/1 Knapsack).
- Sequence comparison and alignment problems (LCS, edit distance, bioinformatics).
- Problems requiring an optimal ordering or grouping of operations (Matrix Chain Multiplication, optimal binary search trees).

12.2 Greedy Algorithms

Strengths:

- Simple to design and implement once a valid ranking rule is identified.
- Typically much faster and more memory-efficient than DP, often $O(n \log n)$.
- Easy to reason about and explain, since the algorithm's behaviour at each step is transparent.

Limitations:

- Only correct for problems where the greedy-choice property genuinely holds; using greedy without proving this can silently produce a suboptimal answer.
- No mechanism to reconsider or backtrack on earlier decisions, so a single poor early choice cannot be corrected later.
- Identifying the correct ranking criterion is not always obvious, and an incorrect ranking rule can produce plausible-looking but wrong results.

Typical Use Cases:

- Interval scheduling and resource-allocation problems with a single shared, non-divisible resource but divisible or count-based objectives (Activity Selection).
- Encoding and compression problems with a provable exchange-argument structure (Huffman Coding).
- Graph construction problems where local edge-weight decisions provably build toward a global minimum structure (Prim's and Kruskal's algorithms for Minimum Spanning Trees).

Strengths / Limitations Matrix		
	Dynamic Programming	Greedy Algorithm
Strengths	Exact optimal solution; handles overlapping subproblems efficiently	Simple, fast, low memory; easy to implement and reason about for suitable problems
Limitations	Higher memory/time for large state spaces; needs well-defined recurrence	Optimal only when greedy-choice property holds; can converge to suboptimal solutions

Figure 8. Strengths and Limitations Matrix

Side-by-side summary of the strengths and limitations discussed above for Dynamic Programming and Greedy algorithms.

NOTE FOR STUDENT REVIEW

This is the primary BL5 (Evaluation) section of the draft. Add your own judgement here: for a specific problem type from your coursework, state which strategy you would choose and justify the choice using the strengths and limitations above, rather than simply restating them.

13. Applications and Synthesis

13.1 Applications by Domain

- Scheduling: CPU process scheduling and job-shop scheduling use DP for weighted or precedence-constrained variants, and Greedy (earliest-deadline-first, earliest-finish-time) for simpler unweighted variants.
- Routing: shortest-path and vehicle-routing problems draw on DP for problems with additional constraints (such as time windows) and on greedy-flavoured algorithms (such as Dijkstra's algorithm, a greedy method with priority-queue-based edge relaxation) for simpler shortest-path cases.
- Resource allocation: budget allocation across competing projects is a natural Knapsack-style problem, DP when allocations are indivisible, Greedy when they are divisible.
- Networking: minimum spanning tree algorithms (Prim, Kruskal) are used directly for designing low-cost network topologies that connect all required nodes.
- Compression: Huffman Coding is used directly in file-compression formats and multimedia codecs to assign shorter codes to more frequent symbols.
- Finance: portfolio selection under a budget constraint mirrors the Knapsack problem; DP is used when investment units are indivisible, greedy ratio-based heuristics are used for quick approximations when units are effectively divisible.
- Inventory: inventory replenishment and cutting-stock problems use DP formulations closely related to Rod Cutting and Coin Change.

- Bioinformatics: DNA and protein sequence alignment rely on LCS-style dynamic programming to measure similarity between sequences.

13.2 Scalability and Algorithm Selection

As problem size grows, the practical choice between DP and Greedy is driven by three questions: whether the problem exhibits overlapping subproblems (favouring DP), whether a ranking rule with a provable greedy-choice property exists (favouring Greedy), and how large the numeric parameters of the problem are relative to the number of items, since pseudo-polynomial DP algorithms can become impractical when those parameters are large even for small n . In practice, many production systems use a greedy algorithm as a fast approximate or exact solution where it is provably valid, and reserve DP (or more expensive exact methods such as integer programming) for the specific subset of cases where greedy is known to fail.

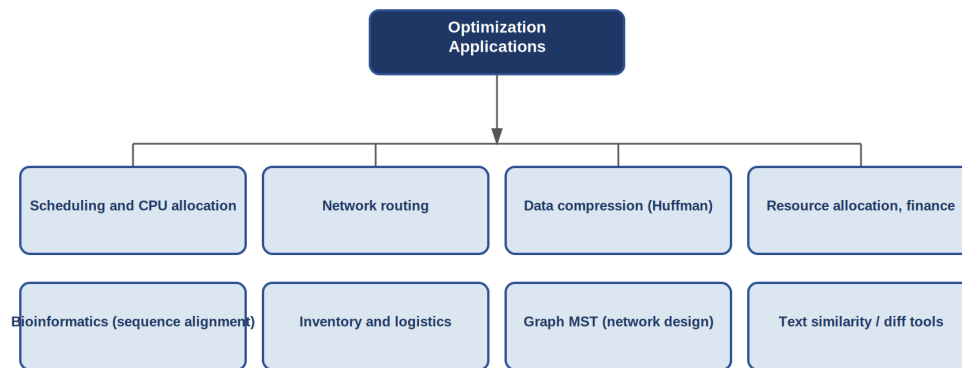


Figure 9. Real-World Application Map for Optimization Techniques

Optimization techniques covered in this draft underpin a wide range of practical domains, from scheduling and networking to compression and bioinformatics.

13.3 Algorithm-Selection Decision Tree

The decision tree below summarizes, at a high level, the reasoning path developed across this document for choosing between Dynamic Programming and a Greedy algorithm for a new optimization problem.

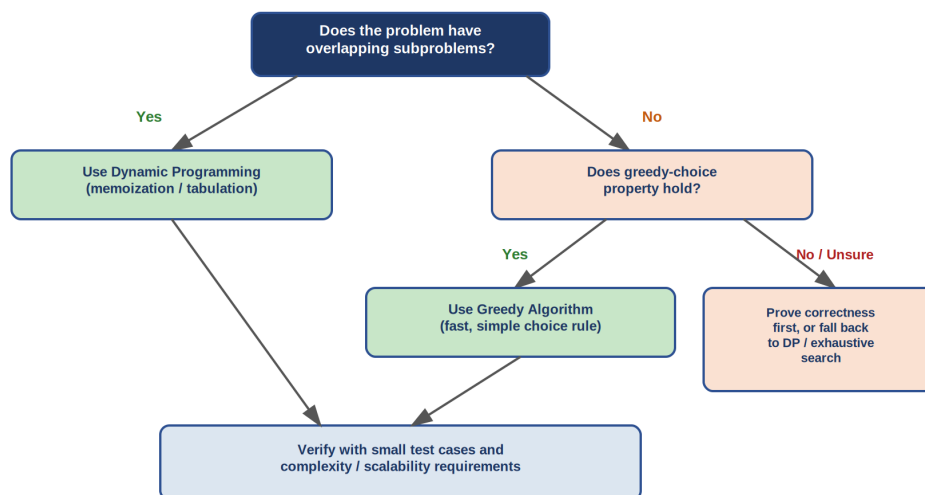


Figure 10. Algorithm-Selection Decision Tree

A structured decision path: check for overlapping subproblems first, then check the greedy-choice property, and verify the chosen approach against small test cases.

14. Conclusion and References

14.1 Synthesis

Across the worked examples in this draft, a consistent pattern emerges: Dynamic Programming and Greedy algorithms are not competing solutions to the same problem but are matched to different problem structures. Dynamic Programming is required whenever a problem has overlapping subproblems and no provable greedy-choice property, as demonstrated by 0/1 Knapsack, which needed a full DP table to guarantee its optimal value of 10. The comparison with Fractional Knapsack, which reached a higher value of 10.75 on the identical item set using only a ratio-based greedy rule, shows that divisibility of items — not the ranking idea itself — is what makes a greedy strategy provably valid. Greedy algorithms are therefore preferable whenever the greedy-choice property can be proven, as in Fractional Knapsack and Activity Selection, because they reach a guarantee of optimality at a fraction of the time and memory cost that Dynamic Programming would require.

14.2 Key Insights

- Optimal substructure is necessary for both strategies, but only Greedy additionally requires the greedy-choice property.
- Overlapping subproblems are the specific signal that DP's memoization/tabulation will pay off; their absence is one reason Greedy can be safely applied without redundant computation.
- Relaxing or tightening a single constraint (such as item divisibility) can change which strategy is correct for what otherwise looks like the same problem, as shown directly by the Knapsack comparison in Section 7.
- Correctness proofs differ in kind: DP correctness is established inductively through the recurrence and base cases, while Greedy correctness is established through an exchange argument on the greedy-choice property.
- Complexity trade-offs are real and measurable: the worked examples showed DP requiring $O(nW)$ time against Greedy's $O(n \log n)$ for closely related problems, a gap that widens as numeric parameters grow.

14.3 Conclusion

Neither Dynamic Programming nor Greedy algorithms are universally superior; the appropriate choice depends entirely on whether the specific optimization problem being solved possesses the greedy-choice property in addition to optimal substructure. Dynamic Programming provides a general-purpose, always-correct approach at higher computational cost, while Greedy provides a narrow but highly efficient approach that is only valid where it can be formally justified. Recognizing which category a new problem falls into, using the diagnostic questions and decision tree developed in this draft (Sections 9 and 13), is the central practical skill this comparison is intended to support.

14.4 References

The following standard algorithms textbooks and resources cover the topics discussed in this draft and are suitable starting points for further reading and verification:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press. (Chapters on Dynamic Programming and Greedy Algorithms.)
- Kleinberg, J., & Tardos, E. Algorithm Design. Pearson/Addison-Wesley. (Chapters on Greedy Algorithms and Dynamic Programming.)

- Levitin, A. Introduction to the Design and Analysis of Algorithms. Pearson. (Chapters on the Greedy Technique and Dynamic Programming.)
- Sedgewick, R., & Wayne, K. Algorithms. Addison-Wesley. (Sections on graph algorithms including Prim's and Kruskal's algorithms.)
- Course lecture notes and slides for CSA06 — Design and Analysis of Algorithms (SIMATS ENGINEERING).